

INTRODUCTION TO PROGRAMMING LANGUAGE

1. How Does a Computer Work?

A computer is an electronic machine. It performs four basic operations: Input, Processing, Storage, and Output.

Computer Workflow:

Input → Processing → Storage → Output

Example:

You type: $5 + 10$

↓

Computer processes the operation

↓

Answer = 15

Think of a computer as a very fast but obedient worker that only follows instructions.

2. What Language Does a Computer Understand?

Computers understand only Binary Language (0 and 1).

0 = OFF

1 = ON

Example:

101010101100110011110000

Humans find binary difficult to read and write, so easier programming languages were developed.

3. How Did Humans Program Computers Earlier?

In the early days, programmers wrote instructions directly in Machine Language.

Example:

101100001110000100011100

Problems:

- Difficult to read

- Difficult to write
- Easy to make mistakes
- Hard to debug
- Time-consuming

4. Assembly Language (Low-Level Language)

Assembly Language introduced mnemonics:

MOV A, B

ADD A, C

Examples:

MOV = Move

ADD = Add

SUB = Subtract

MUL = Multiply

5. Problems with Assembly Language

Assembly Language is machine dependent.

Programs written for one processor may not run on another.

Problems:

- Hard to learn
- Difficult for large programs
- Time-consuming
- Different processors use different instructions

6. Introduction of High-Level Languages

High-level languages are easier for humans.

Example:

```
sum = a + b;
```

Computers cannot understand this directly, so a translator is needed.

7. Who Translates High-Level Language?

Compiler Translation Process:

Source Code → Compiler → Machine Code → Execution → Output

Example:

```
printf("Hello");
```

8. Why Was C Language Introduced?

Dennis Ritchie developed C in 1972 at Bell Labs.

Goals:

- Easier than Assembly
- Faster execution
- Hardware interaction
- Operating System development

Unix was largely written in C.

9. Why Is C Called a Middle-Level Language?

High-Level Features:

`a = b + c;`

Low-Level Features:

Memory access using pointers.

Assembly $\leftarrow$ C $\rightarrow$ High-Level Languages

10. Features of C Language

- Simple
- Fast
- Portable
- Structured
- Rich Library Functions
- Supports Pointers
- Efficient Memory Usage
- Foundation for Other Languages

11. How a C Program Works

Programmer Writes C Code

↓

Compiler Translates Code

↓

Machine Code Generated

↓

Computer Executes Code

↓

Output Displayed

History of C Language (Complete Story)

Machine Language → Assembly Language → BCPL (1966) → B Language (1969) → C Language (1972)

1978: K&R C

1989: ANSI C (C89)

1990: C90

1999: C99

2011: C11

2018: C17

2023: C23

Basic Structure of a C Program

```
#include <stdio.h>
```

```
int main()  
{  
    statements;  
}
```

Components:

1. Header File
2. Main Function
3. Braces
4. Statements

Part 1: Header File

```
#include <stdio.h>
```

here stdio.h stands for - standard input output header file

So by writing include <stdio.h> we Include Standard Input Output Library

It provides functions like:

```
printf()  
scanf()
```

Think of it as bringing tools into your toolbox before starting work.

Part 2: Main Function

```
int main()
```

Every C program starts execution from main().

Think of it as:

Main Gate of the Program

The computer enters here first.

Part 3: Opening and Closing Braces

```
{  
}
```

Everything inside belongs to the main function.

Example:

```
int main()  
{  
    statements  
}
```

Part 4: Statements

Instructions given to the computer.

Example:

```
printf("Hello World");
```

Every statement ends with:

```
;
```

This semicolon acts like a full stop in English.